Design and Analysis of Algorithms
AIML, 5TH SEM

CSEL30503MJ

Module - 1

Divide And Conquer Method: Binary Search, Merge Sort, Quick sort, Heap sort and Strassen's matrix multiplication algorithms.

Content

1. Binary search
2. Merge Sort
3. Quick sort
4. Heap sort
5. Strassen's matrix multiplication algorithms

Divide and Conquer Method: Binary Search

Concept: Binary Search is an efficient algorithm for finding an item from a sorted list of items. It works by repeatedly dividing the search interval in half.

Example: Let's search for the number 23 in the sorted array
A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91].

1. **Initial:** low = 0, high = 9. mid = (0+9)/2 = 4. A[4] = 16. Since 16 < 23, search in the right half.
2. **Iteration 1:** low = mid + 1 = 5, high = 9. mid = (5+9)/2 = 7. A[7] = 56. Since 56 > 23, search in the left half.
3. **Iteration 2:** low = 5, high = mid - 1 = 6. mid = (5+6)/2 = 5. A[5] = 23. Since 23 == 23, found! Return index 5.

Divide and Conquer Method: Binary Search

Recurrence Relation: Let T(n) be the time complexity to search in an array of size n.

Divide: The array is divided into two halves. This takes O(1) time.

Conquer: We only recurse on one of the two halves. This takes T(n/2) time.

Combine: No combining step is needed. This takes O(1) time.

So, the recurrence relation is: $T(n)=T(n/2)+O(1)$ Base case: $T(1)=O(1)$ (when the array has only one element, we check it directly).

Divide and Conquer Method: Binary Search

- **Solving the Recurrence Relation (using Master Theorem):** The Master Theorem is applicable here. For T(n)=aT(n/b)+f(n):

  a=1 (one subproblem)

  b=2 (problem size is halved)

  f(n)=O(1)

  Compare f(n) with $n^{\log_b a}$: $n^{\log_2 1}=n^0=1$. Since f(n)=O(1)=Θ($n^{\log_b a}$), this falls into **Case 2** of the Master Theorem.

  Case 2 states that if f(n)=Θ($n^{\log_b a}$), then T(n)=Θ($n^{\log_b a}$ logn). So, T(n)=Θ(1·logn)=Θ(logn).

  Time Complexity: O(logn)

Divide and Conquer Method: Merge Sort

- **Concept:** Merge Sort is a comparison-based sorting algorithm. It works by recursively dividing the unsorted list into n sub lists, each containing one element (a list of one element is considered sorted). Then, it repeatedly merges sub lists to produce new sorted sub lists until there is only one sorted list remaining.
- Example: Sort the array A = [38, 27, 43, 3, 9, 82, 10].
  - Divide:
    - [38, 27, 43, 3], [9, 82, 10]
    - [38, 27], [43, 3], [9, 82], [10]
    - [38], [27], [43], [3], [9], [82], [10] (Base case: single elements)

Divide and Conquer Method: Merge Sort

- Conquer (Merge):
  - Merge [38] and [27] -> [27, 38]
  - Merge [43] and [3] -> [3, 43]
  - Merge [9] and [82] -> [9, 82]
  - Merge [10] -> [10] (no merge needed for single element)
- Merge (continued):
  - Merge [27, 38] and [3, 43] -> [3, 27, 38, 43]
  - Merge [9, 82] and [10] -> [9, 10, 82]
- Final Merge:
  - Merge [3, 27, 38, 43] and [9, 10, 82] -> [3, 9, 10, 27, 38, 43, 82]

Divide and Conquer Method: Merge Sort

Recurrence Relation: Let T(n) be the time complexity to sort an array of size n.

Divide: The array is divided into two halves. This takes O(1) time.

Conquer: Recursively sort the two halves. This takes 2T(n/2) time.

Combine: Merging the two sorted halves back into a single sorted array takes O(n) time (as it involves iterating through all n elements once).

So, the recurrence relation is: $T(n)=2T(n/2)+O(n)$ Base case: $T(1)=O(1)$ (an array with one element is already sorted).

Divide and Conquer Method: Merge Sort

Solving the Recurrence Relation (using Master Theorem): For T(n)=aT(n/b)+f(n):

- a=2
- b=2
- f(n)=O(n)

Compare f(n) with $n^{\log_b a}$: $n^{\log_2 2}$ =n^1= n. Since f(n)=O(n)=Θ(n$\log_b a$), this falls into **Case 2** of the Master Theorem.

Case 2 states that if f(n)=Θ($n^{\log_b a}$), then T(n)=Θ($n^{\log_b a}$ logn). So, T(n)=Θ(nlogn).

- **Time Complexity:** O(nlogn)

Divide and Conquer Method: Quick Sort

Concept: Quick Sort is an efficient, in-place sorting algorithm. It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively.

Example: Sort the array A = [10, 80, 30, 90, 40, 50, 70]. Let's pick the last element as the pivot.

- Choose Pivot: Let's choose 70 as the pivot.
- Partition: Rearrange elements such that all elements smaller than 70 come before it, and all elements larger come after it.
  - Initial: [10, 80, 30, 90, 40, 50, 70]
  - After partitioning (pivot 70 is now at its sorted position): [10, 30, 40, 50, 70, 90, 80]
  - Left sub-array: [10, 30, 40, 50]Right sub-array: [90, 80]

Divide and Conquer Method: Quick Sort

Recursively Sort: Sort [10, 30, 40, 50]

Sort [90, 80]

This process continues until all sub-arrays are sorted.

Recurrence Relation: Let T (n) be the time complexity to sort an array of size n.

Divide: Partitioning the array around a pivot takes O(n) time.

Conquer: Recursively sort the two sub-arrays. The sizes of these sub-arrays depend on the pivot choice.

Best Case: The pivot divides the array into two almost equal halves (e.g., n/2 and n/2).

Worst Case: The pivot is always the smallest or largest element, leading to one sub-array of size $n-1$ and another of size 0.

Average Case: The pivot roughly splits the array into a fraction, e.g., n/k and $(1-1/k)n$.

Combine: No explicit combining step is needed, as sorting is done in-place. This takes O(1) time.

Divide and Conquer Method: Quick Sort

1. **Worst-Case Recurrence Relation:** If the partition is maximally unbalanced (e.g., pivot is always the smallest/largest element):

   $T(n)=T(n-1)+T(0)+O(n)$

   $T(n)=T(n-1)+O(n)$

   Base case: $T(1)=O(1)$

- **Solving Worst-Case Recurrence (using Back Substitution):** $T(n)=T(n-1)+cn$

  $T(n)=(T(n-2)+c(n-1))+cn=T(n-2)+c(n-1)+cn$
  $T(n)=(T(n-3)+c(n-2))+c(n-1)+cn=T(n-3)+c(n-2)+c(n-1)+cn$

  …

  $T(n) = T(1) + c\sum_{i=2}^{n} i = T(1) + c(2 + 3 + \cdots + n)$

  $T(n) = O(1) + c(\frac{n(n+1)}{2} - 1)=O(1)+O(n^2)$

  So, $T(n)=O(n^2)$.

Divide and Conquer Method: Quick Sort

2. Best-Case Recurrence Relation: If the partition is always perfectly balanced:

T(n)=2T(n/2)+O(n)

Base case: T(1)=O(1)

Solving Best-Case Recurrence (using Master Theorem): This is the same as Merge Sort's recurrence.

a=2,b=2,f(n)=O(n).

$n^{\log_2 2}$=n.

f(n)=Θ($n^{\log_b a}$), so Case 2.

Solution: T(n)=Θ(nlogn).

Divide and Conquer Method: Quick Sort

3. Average-Case Recurrence Relation: The average-case analysis is more complex, but it can be shown that if pivots are chosen randomly, the expected time complexity is also O(nlogn). One common way to model this is:

$T(n)=\frac{1}{n}\sum_{k=0}^{n-1}(T(k)+T(n-1-k))+O(n)$

This simplifies to $T(n)\approx\frac{2}{n}\sum_{k=n/2}^{n-1}T(k)+O(n)$ (ignoring smaller terms)

This recurrence, when solved, yields T(n)=O(nlogn).

Time Complexity:

- **Worst Case:** $O(n^2)$
- **Best Case:** O(nlogn)
- **Average Case:** O(nlogn)

Divide and Conquer Method: Heap Sort

Concept: Heap Sort is an in-place comparison-based sorting algorithm. It typically involves two phases:

1. **Build Max-Heap:** Transforms the input array into a max-heap (a complete binary tree where each parent node is greater than or equal to its children).
2. **Extract Max & Heapify:** Repeatedly extracts the maximum element (the root), moves it to the end of the array, and then re-heapifies the remaining elements.

Divide and Conquer Method: Heap Sort

Example: (Covered in previous detailed explanation, but briefly)

Sort A = [4, 1, 3, 2, 16, 9, 10].

1. Build Max-Heap:
   - Original: [4, 1, 3, 2, 16, 9, 10]
   - After BUILD-MAX-HEAP: [16, 9, 10, 2, 4, 1, 3] (conceptual heap structure)
2. Extract Max & Heapify:
   - Swap 16 (root) with 3 (last element). Array: [3, 9, 10, 2, 4, 1, **16**] (16 is sorted)
   - Heapify [3, 9, 10, 2, 4, 1]: [10, 9, 3, 2, 4, 1]
   - Swap 10 with 1. Array: [1, 9, 3, 2, 4, **10**, **16**] (10, 16 sorted)
   - Heapify [1, 9, 3, 2, 4]: [9, 4, 3, 2, 1]...and so on, until the array is sorted.

Divide and Conquer Method: Heap Sort

- **Recurrence Relation**: Heap Sort's analysis is usually done by summing the complexities of its phases, rather than a single divide-and-conquer recurrence relation like Merge Sort or Quick Sort. This is because its recursive calls are implicit within the MAX-HEAPIFY function, which is called iteratively.
- Let T(N) be the total time complexity.
- T(N)=T build-heap (N)+T extract-max (N)

Divide and Conquer Method: Heap Sort

1. BUILD-MAX-HEAP:

- This phase involves calling MAX-HEAPIFY on N/2 nodes. While a single MAX-HEAPIFY takes O(logk) for a heap of size k, a tighter analysis sums up the costs for all nodes in the heap.
- The complexity is known to be O(N).
- A 'recurrence' for MAX-HEAPIFY itself is often M(h)=M(h−1)+O(1), where h is the height, solving to M(h)=O(h)=O(logk).

2. Extract-Max Phase:

- This phase performs N−1 extraction operations.
- Each extraction involves swapping the root with the last element (O(1)) and then calling MAX-HEAPIFY on the (reduced) heap, which takes O(logk) where k is the current heap size.
- The total cost is $\sum_{k=2}^{N} O(logk) = O(NlogN)$.

Divide and Conquer Method: Heap Sort

- **Solving the Recurrence Relation**:

As explained, it's a sum of costs rather than a traditional recurrence:

$T(N)=O(N)+O(NlogN)$

- **Time Complexity:**

O(NlogN) (dominated by the extraction phase).

Divide and Conquer Method: Strassen's Matrix Multiplication Algorithm

- **Concept:** Strassen's algorithm is a recursive method for matrix multiplication. It reduces the number of multiplications required from 8 (in the standard algorithm) to 7, which improves the asymptotic time complexity. It works for square matrices whose dimensions are powers of 2. For other dimensions, matrices are padded with zeros.

  Example (Conceptual): To multiply two N×N matrices A and B to get C=A×B:

  $C=\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$, $A=\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$, $B=\begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$

  Where A_{ij}, B_{ij}, C_{ij} are N/2×N/2 sub-matrices.

Divide and Conquer Method: Strassen's Matrix Multiplication Algorithm

- Standard matrix multiplication would calculate Cij as:

$C_{11}=A_{11}B_{11}+A_{12}B_{21}$

$C_{12}=A_{11}B_{12}+A_{12}B_{22}$

$C_{21}=A_{21}B_{11}+A_{22}B_{21}$

$C_{22}=A_{21}B_{12}+A_{22}B_{22}$

- This requires 8 multiplications of N/2×N/2 matrices and 4 additions of N/2×N/2 matrices.

Divide and Conquer Method: Strassen's Matrix Multiplication Algorithm

- Strassen's algorithm uses 7 multiplications (M1 to M7) and more additions/subtractions to compute the Cij blocks.

1. P_1=$A_{11}(B_{12}-B_{22})$
2. P_2=$(A_{11}+A_{12})B_{22}$
3. P_3=$(A_{21}+A_{22})B_{11}$
4. P_4=$A_{22}(B_{21}-B_{11})$
5. P_5=$(A_{11}+A_{22})(B_{11}+B_{22})$
6. P_6=$(A_{12} - A_{22})(B_{21}+B_{22})$
7. P_7=$(A_{11} - A_{21})(B_{11}+B_{12})$

Then, the C_{ij} blocks are calculated using these P_k values:

$C_{11}= P_5+ P_4 - P_2 + P_6$

$C_{12}= P_1 + P_2$

$C_{21}= P_3 + P_4$

$C_{22}= P_5+ P_1 - P_3 - P_7$

Divide and Conquer Method: Strassen's Matrix Multiplication Algorithm

Recurrence Relation: Let T(N) be the time complexity to multiply two N×N matrices.

Divide: Dividing the matrices into sub-matrices, and performing additions/subtractions to compute the inputs for P_k takes $O(N^2)$ time (since adding/subtracting N/2×N/2 matrices takes $(N/2)^2=N^2/4$ time, and there are several such operations).

Conquer: Recursively perform 7 multiplications of N/2×N/2 matrices. This takes 7T(N/2) time.

Combine: Performing additions/subtractions to compute the final Cij blocks from Pk values also takes $O(N^2)$ time.

So, the recurrence relation is: $T(N)=7T(N/2)+O(N^2)$

Base case: T(1)=O(1) (multiplying 1×1 matrices is constant time).

Divide and Conquer Method: Strassen's Matrix Multiplication Algorithm

- **Solving the Recurrence Relation (using Master Theorem):** For $T(N)=aT(N/b)+f(N)$:

  $a=7$

  $b=2$

  $f(N)=O(N^2)$

  Compare f(N) with $N^{\log_b a}$: $N^{\log_2 7}$. Since $\log_2 7 \approx 2.807$.

  We have $N^{\log_2 7}$ vs N^2. Since $2<2.807$, we have $N^2=O(N^{\log_2 7-\epsilon})$ for some $\epsilon>0$. This falls into **Case 1** of the Master Theorem.

  Case 1 states that if $f(N)=O(N^{\log_2 7-\epsilon})$, then $T(N)=\Theta(N^{\log_b a})$. So, $T(N)=\Theta(N^{\log_2 7})$.

Time Complexity: $O(N^{\log_2 7})\approx O(N^{2.807})$ This is an improvement over the standard matrix multiplication algorithm which has a time complexity of $O(N^3)$.

Suggestive Questions

Q1. What is the recurrence relation for the standard Binary Search algorithm?

Q2. Is Merge Sort an in-place sorting algorithm? Justify your answer in one sentence.

Q3. What is the worst-case time complexity of Quick Sort? What specific condition causes this complexity?

Q4. State the number of recursive multiplications performed in Strassen's algorithm to multiply two n×n matrices.

Q5. Is Heap Sort a stable sorting algorithm? Briefly explain why.

Suggestive Questions

Q6. Derive the time complexity of a recursive algorithm that divides a problem of size n into five subproblems of size n/5 and performs O(n) work to divide and combine the results. Use the Master Theorem to justify your answer.

Q7. Explain the concept of a "stable sort" with respect to Merge Sort. Provide a brief step-by-step trace of Merge Sort on the array $[5_a,2,8,5_b,1,6]$ to demonstrate its stability (where 5_a and 5_b are identical values).

Q8. Write the recurrence relation for Quick Sort in the best-case scenario. Solve this recurrence relation and state the resulting time complexity.

Q9. Describe the two main phases of the Heap Sort algorithm. For each phase, provide its time complexity and explain the reason for it.

Suggestive Questions

10. For the recurrence relation $T(n)=3T(n/4)+n^2$, derive its time complexity using the Master Theorem. Show all steps of your calculation and clearly state which case of the Master Theorem applies.
11. Consider a divide and conquer algorithm that splits a problem of size n into four subproblems of size n/2, with the division and combination step taking O(n) time. Write its recurrence relation and solve it using the Recursion Tree Method. Draw the first three levels of the tree, calculate the work done at each level, and sum it to find the total time complexity.
12. The standard matrix multiplication algorithm has a recurrence relation of $T(n)=8T(n/2)+O(n^2)$, which solves to $O(n^3)$. Strassen's algorithm provides a significant improvement. Derive the recurrence relation for Strassen's algorithm, explaining the source of each term.
13. Solve the recurrence relation for Strassen's algorithm using the Master Theorem.